



UNIDADE I

Modelagem de Banco de Dados e NoSQL

Profa. Dra. Vanessa Lessa

O Que é Modelagem Relacional?

- A modelagem de banco de dados relacional estabelece a estrutura lógica para armazenamento, acesso e manipulação de dados. Ela representa de forma abstrata os elementos do mundo real, garantindo **integridade, consistência e eficiência**.
- Proposto por **Edgar F. Codd na década de 1970**, o modelo relacional organiza dados em **tabelas (relações)** compostas por **linhas (tuplas)** e **colunas (atributos)**. Cada tabela representa uma entidade; cada tupla, uma instância; e cada atributo, uma característica dessa entidade.

Elementos Fundamentais do Modelo Relacional

Tabelas

- Representam entidades do mundo real (clientes, produtos, pedidos).
Cada tabela é uma relação.

Chave Primária

- Identifica unicamente cada registro, evitando duplicidades.
Base para relacionamentos.

Chave Estrangeira

- Referencia a chave primária de outra tabela, garantindo a integridade referencial.

Normalização

- Regras que eliminam redundâncias e dependências desnecessárias, promovendo consistência.

A modelagem passa por três etapas:

- modelo conceitual (DER);
- modelo lógico (estrutura relacional); e
- modelo físico (implementação no SGBD).

A Pirâmide DIKW

Do Dado à Sabedoria

- A pirâmide DIKW representa os níveis crescentes de valor cognitivo:

**Dado → Informação →
Conhecimento → Sabedoria.**

- Cada nível agrega valor ao anterior e exige maior envolvimento humano.
- Bancos de dados lidam com dados e informações, mas sua eficácia está ligada à geração de conhecimento e sabedoria organizacional.



Os Quatro Níveis da Pirâmide DIKW

Dado

- Elemento bruto, sem contexto.
- Ex.: “35”, “Maria”, “azul”.

Informação

- Dado contextualizado.
- Ex.: “Maria tem 35 anos”.

Conhecimento

- Informação aplicada com experiência e raciocínio crítico.

Sabedoria

- Uso prudente do conhecimento com discernimento ético e visão estratégica.

Entidades: Definição e Tipos

- Entidades representam objetos ou conceitos do mundo real sobre os quais o sistema armazena informações. Podem ser **concretas** (pessoas, produtos) ou **abstratas** (contratos, matrículas).

Entidades Fortes

- Existência independente, identificadas por atributos próprios.
- Ex.: Cliente, Produto.
- Possuem chave primária própria.

Entidades Fracas

- Dependem de outra entidade para existir.
- Ex.: Item de Pedido depende de Pedido.
- Sua chave primária combina a chave da entidade forte com atributo adicional.

Atributos: Classificação e Tipos

Simple

- Indivisíveis. Ex.: CPF.

Compostos

- Subdivididos. Ex.: Endereço → Rua, Número, Bairro.

Multivalorados

- Múltiplos valores. Ex.: Telefones.

Derivados

- Calculados. Ex.: Idade ← Data de Nascimento.

Nulos

- Podem não ter valor atribuído.

Chave Primária: Natural vs. Artificial

Critérios para uma boa chave primária

- Unicidade: identifica cada ocorrência de forma única.
- Estabilidade: não deve mudar ao longo do tempo.
- Simplicidade: preferencialmente um único atributo.
- Não nula: nunca pode assumir valor nulo.

Natural

- Atributo real com significado próprio.
- Ex.: CPF, número de matrícula.
- Pode ter desvantagens como mudanças de valor ou restrições legais.

Artificial

- Código criado para identificação, geralmente numérico sequencial.
- Amplamente usada pela simplicidade e estabilidade.
- Alterar o valor de uma chave primária pode gerar efeitos em cascata em diversas tabelas relacionadas.

Relacionamentos entre Entidades

- Relacionamentos expressam como os objetos do domínio se conectam.
- No modelo relacional, são implementados por meio de **chaves estrangeiras**, garantindo a integridade referencial.

Relacionamento 1:1

- Cada instância de A está associada a, no máximo, uma de B.
- Ex.: Campus $\leftrightarrow$ Diretor.

Relacionamento 1:N

- Uma instância de A se relaciona com várias de B.
- Ex.: Departamento $\rightarrow$ Funcionários.

Relacionamento N:N

- Múltiplas instâncias de A associadas a múltiplas de B.
- Ex.: Alunos $\leftrightarrow$ Disciplinas.
- Exige tabela associativa.

Participação e Integridade Referencial

Participação Obrigatória

- Toda ocorrência da entidade deve estar associada a outra.
- Implementada com chaves estrangeiras **não nulas**.
- Ex.: Aluno deve estar vinculado a um Curso.

Participação Opcional

- A entidade pode existir sem associação imediata.
 - Permite **valores nulos** na chave estrangeira.
 - Ex.: Curso pode existir sem alunos matriculados.
-
- A **integridade referencial** garante que toda chave estrangeira corresponda a uma chave primária existente, evitando registros órfãos e dados inconsistentes.

Relacionamentos Ternários e Entidades Associativas

Relacionamentos Ternários

- Envolvem três entidades simultaneamente. Ex.: Funcionário + Projeto + Papel.
- O papel desempenhado depende da combinação específica das três entidades.
- Não pode ser representado por dois relacionamentos binários independentes sem perda de informação.

Entidade Associativa

- Representa explicitamente a relação N:N como uma nova entidade.
- Ex.: Matrícula (Aluno $\leftrightarrow$ Disciplina).
- Armazena semestre, nota e situação. Mantém coerência do modelo e facilita consultas complexas.
- A transformação de relacionamentos N:N em entidades associativas é um exemplo direto de como a modelagem conceitual influencia a estrutura relacional.

O Diagrama Entidade-Relacionamento (DER)

- Ferramenta de modelagem conceitual que representa graficamente entidades, atributos e relacionamentos, fornecendo visão clara da estrutura dos dados antes da implementação.
- **Retângulos:** Representam entidades do sistema.
- **Elipses:** Representam atributos das entidades.
- **Losangos:** Representam relacionamentos entre entidades.
- **Sublinhado:** Destaca atributos identificadores (chave primária).
- **Exemplo de DER:** A entidade Alunos ilustra os diferentes tipos de atributos:
 - id_aluno (chave primária)
 - nome (simples)
 - numero_telefone (multivalorado)
 - data_nascimento (simples)
 - endereço (composto: rua, número, cidade, estado)
 - nro_passaporte (nulo)
 - idade (derivado de data_nascimento)

Processo de Criação do DER

Entendimento

- Entrevistas e análise de documentos.

Entidades

- Identificar objetos com significado próprio.

Atributos

- Definir características e chaves primárias.

Relacionamentos

- Cardinalidade e participação entre entidades.
- A criação do DER é um processo iterativo: o modelo é constantemente revisado à medida que o entendimento do domínio se aprofunda. Validações frequentes com usuários reduzem erros e garantem fidelidade ao negócio.

Boas Práticas na Criação do DER

Nomenclatura Clara

- Use substantivos no singular para entidades e terminologia do negócio.
- Evite nomes genéricos ou ambíguos.

Nível de Detalhamento Adequado

- Nem excessivamente detalhado nem simplificado demais.
- Equilíbrio é essencial.

Organização Visual

- Distribua entidades de forma lógica, minimize cruzamentos de linhas e use espaçamento adequado para legibilidade.

Documentação Complementar

- Registre decisões tomadas, justificativas e resultados das validações com usuários para referência futura.

Interatividade

Analise as afirmações:

- I. A chave primária tem como função identificar unicamente cada registro de uma tabela, não podendo assumir valores nulos, sendo fundamental para o estabelecimento de relacionamentos entre entidades.
- II. Relacionamentos do tipo N:N podem ser implementados diretamente entre duas tabelas sem a necessidade de uma tabela intermediária, desde que ambas possuam chave primária artificial.
- III. O Diagrama Entidade-Relacionamento (DER) é utilizado na fase de modelagem conceitual e representa graficamente entidades, atributos e relacionamentos antes da implementação física do banco de dados.
- IV. Entidades fracas possuem existência independente e sempre apresentam chave primária formada por um único atributo, não dependendo de nenhuma outra entidade do modelo.

Interatividade

Está correto apenas o que se afirma em:

- a) I e III.
- b) I e II.
- c) II e IV.
- d) III e IV.
- e) I e IV.

Resposta

Analise as afirmações:

- I. A chave primária tem como função identificar unicamente cada registro de uma tabela, não podendo assumir valores nulos, sendo fundamental para o estabelecimento de relacionamentos entre entidades.
- II. Relacionamentos do tipo N:N podem ser implementados diretamente entre duas tabelas sem a necessidade de uma tabela intermediária, desde que ambas possuam chave primária artificial.
- III. O Diagrama Entidade-Relacionamento (DER) é utilizado na fase de modelagem conceitual e representa graficamente entidades, atributos e relacionamentos antes da implementação física do banco de dados.
- IV. Entidades fracas possuem existência independente e sempre apresentam chave primária formada por um único atributo, não dependendo de nenhuma outra entidade do modelo.

Está correto apenas o que se afirma em:

- a) I e III.

O que é Normalização?

- A normalização é um conjunto de princípios que orienta a organização das tabelas de forma lógica, consistente e livre de redundâncias desnecessárias. Seu objetivo central é estruturar os dados de modo que cada informação seja armazenada no local mais apropriado, evitando anomalias de inserção, atualização e exclusão.
- O processo baseia-se na análise das dependências funcionais entre atributos e na aplicação progressiva de regras chamadas formas normais. Cada forma normal exige que a anterior já tenha sido satisfeita.
- **As Três Primeiras Formas Normais**
 - 1FN - Valores atômicos, sem grupos repetitivos.
 - 2FN - Elimina dependências parciais da chave composta.
 - 3FN - Elimina dependências transitivas entre atributos.
- A aplicação é progressiva: para atingir a 3FN, a tabela deve obrigatoriamente satisfazer 1FN e 2FN. Cada nível adiciona organização e controle ao modelo relacional.

Primeira Forma Normal (1FN)

- Uma tabela está na 1FN quando todos os seus atributos possuem valores atômicos – indivisíveis – e não existem grupos repetitivos ou atributos multivalorados em uma mesma coluna. Cada célula deve conter um único valor.
- Violação típica: Múltiplos telefones separados por vírgula em um único campo.
- Solução: Criar tabela separada para telefones, vinculada por chave estrangeira.

1FN: Atomicidade e Identificação Única

Atomicidade

- Cada atributo armazena um único valor por registro. Consultas tornam-se mais precisas e atualizações menos propensas a erros.

Identificação única

- Embora a definição formal da 1FN não exija explicitamente uma chave primária, a identificação única dos registros é requisito implícito para o funcionamento adequado no modelo relacional. Sua ausência gera ambiguidades e dificulta a aplicação das formas normais subsequentes.

Segunda Forma Normal (2FN)

- A 2FN trata das dependências parciais: um atributo não chave depende apenas de parte da chave primária composta, e não da chave como um todo. Para estar na 2FN, todos os atributos não chave devem depender da chave inteira.
- A 2FN só se aplica a tabelas com chaves primárias compostas. Tabelas com chave simples que já estão na 1FN automaticamente satisfazem a 2FN.
- **Exemplo Clássico – Matrículas**
 - Matrículas.
 - Disciplinas.
 - Alunos.
- O nome do aluno depende apenas do aluno_id; o nome da disciplina, apenas do disciplina_id. A decomposição elimina dependências parciais e reduz redundância.

Benefícios da 2FN

Menos redundância

- Nome do aluno armazenado uma única vez, não repetido em cada matrícula.

Atualizações simples

- Alteração em um atributo feita em apenas um local, sem risco de divergência.

Tabelas coesas

- Cada tabela representa um único conceito com responsabilidade bem definida.
- A eliminação de dependências parciais também previne anomalias de inserção e exclusão: não é mais necessário ter um pedido para cadastrar um produto, nem a exclusão de uma matrícula apaga dados do aluno.

Terceira Forma Normal (3FN)

- A 3FN elimina dependências transitivas: quando um atributo não chave depende de outro atributo não chave, que, por sua vez, depende da chave primária. Todos os atributos não chave devem depender direta e exclusivamente da chave primária.

- Exemplo: na tabela de funcionários,

$\text{cod_funcionario} \rightarrow \text{cod_departamento} \rightarrow \text{nome_departamento}$.

O nome do departamento depende transitivamente do funcionário.

- **3FN: Decomposição de Funcionários**

- Tabela original

$\text{cod_funcionario}, \text{nome}, \text{cod_departamento}, \text{nome_departamento}, \text{local_departamento}$

- Funcionários

$\text{cod_funcionario}, \text{nome}, \text{cod_departamento}$

- Departamentos

$\text{cod_departamento}, \text{nome_departamento}, \text{local_departamento}$

- Com a separação, qualquer alteração no nome ou local do departamento é feita em um único registro da tabela Departamentos, eliminando inconsistências e anomalias de exclusão.

Exemplo Integrado: Pedidos de Clientes

Considere uma tabela inicial com:

- id_pedido, data, nome_cliente, endereço, telefone, cod_produto, descrição_produto, quantidade, preço_unitário

Aplicar 1FN

- Um produto por linha; eliminar grupos repetitivos de produtos no pedido.

Aplicar 2FN

- Separar em: Pedidos, Clientes, Produtos e Itens de Pedido.

Aplicar 3FN

- Separar Cidades da tabela de Clientes, eliminando dependência transitiva via código da cidade.

As Três Anomalias Clássicas

Anomalia de Inserção

- Impossibilidade de cadastrar um dado sem que outro esteja presente – ex.: não é possível registrar um departamento sem ter um funcionário vinculado.

Anomalia de Atualização

- A mesma informação repetida em vários registros exige múltiplas atualizações. Se alguma falhar, o banco passa a conter dados inconsistentes.

Anomalia de Exclusão

- A remoção de um registro apaga informações que deveriam ser preservadas – ex.: excluir o último pedido de um cliente elimina também seus dados.

Como Cada Forma Normal Combate as Anomalias

1FN

- Elimina grupos repetitivos e atributos multivalorados – resolve problemas de atomicidade que dificultam inserção e atualização.

2FN

- Elimina dependências parciais – reduz redundâncias que geram inconsistências em atualizações e anomalias de inserção/exclusão.

3FN

- Elimina dependências transitivas – previne propagação de redundâncias sutis e anomalias de exclusão mais complexas.

Redução de Redundâncias

- A redundância ocorre quando a mesma informação é armazenada em mais de um local. Quando um dado redundante precisa ser atualizado, todas as suas ocorrências devem ser modificadas de forma sincronizada – caso contrário, o banco passa a conter versões conflitantes da mesma informação.
- A origem da redundância está frequentemente em modelos mal estruturados, em que diferentes conceitos são misturados em uma mesma tabela ou as dependências funcionais não são corretamente compreendidas.

Tipos de Redundância

Redundância explícita

- Atributos de uma entidade repetidos em tabelas de outras entidades. Surge quando relações não são corretamente modeladas, criando dependências implícitas fora do controle do modelo relacional.

Redundância de dados derivados

- Armazenamento de valores que poderiam ser calculados a partir de outros dados já existentes. Sempre que possível, dados derivados devem ser obtidos dinamicamente.

Estratégias Práticas para Reduzir Redundâncias

Separar entidades distintas

- Decompor tabelas que misturam conceitos diferentes, tornando cada uma responsável por um único conceito.

Identificar dependências funcionais

- Analisar quais atributos determinam outros; base para a decomposição correta das tabelas.

Usar chaves primárias e estrangeiras

- Relacionar tabelas por referência, sem duplicar dados; integridade referencial impede registros inválidos.

Padronizar domínios de valores

- Restrições de unicidade e domínios claros evitam registros duplicados que representam o mesmo elemento real.

Normalização vs. Desempenho

O trade-off

- Modelos altamente normalizados podem exigir maior número de junções nas consultas, impactando o desempenho em cenários específicos. No entanto, esse custo, geralmente, é mitigado por índices adequados, enquanto os benefícios de integridade e manutenção costumam superar os custos.

Desnormalização controlada

- Quando necessária, deve ser uma decisão consciente e documentada, com mecanismos que garantam a consistência dos dados redundantes. É exceção, não regra – e a normalização continua sendo o ponto de partida conceitual.

Recomendação geral

- A 3FN oferece equilíbrio adequado entre integridade e simplicidade de acesso. É amplamente adotada como padrão em sistemas transacionais tradicionais.

Impactos Práticos da Normalização

Manutenção simplificada

- Cada informação em um único local; atualizações afetam apenas a tabela responsável pelo dado.

Integridade garantida

- Restrições de chave primária e estrangeira aplicadas com maior eficácia em modelos normalizados.

Flexibilidade evolutiva

- Novos atributos ou entidades incorporados de forma controlada, sem reestruturar grandes porções do modelo.

Auditoria facilitada

- Cada dado rastreável em sua origem; essencial em ambientes que exigem conformidade com normas e regulamentos.

Interatividade

Analise as afirmações:

- I. A Primeira Forma Normal (1FN) exige que todos os atributos possuam valores atômicos, impedindo a existência de grupos repetitivos ou atributos multivalorados em uma mesma coluna.
- II. A Segunda Forma Normal (2FN) elimina dependências transitivas entre atributos não chave, garantindo que todos dependam exclusivamente da chave primária.
- III. A Terceira Forma Normal (3FN) busca eliminar dependências transitivas, evitando que atributos não chave dependam de outros atributos não chave.
- IV. A normalização elimina completamente a necessidade de junções entre tabelas, tornando as consultas sempre mais rápidas e simples.

Interatividade

Está correto apenas o que se afirma em:

- a) I e II.
- b) II e IV.
- c) I, II e III.
- d) I e III.
- e) III e IV.

Resposta

Analise as afirmações:

- I. A Primeira Forma Normal (1FN) exige que todos os atributos possuam valores atômicos, impedindo a existência de grupos repetitivos ou atributos multivalorados em uma mesma coluna.
- II. A Segunda Forma Normal (2FN) elimina dependências transitivas entre atributos não chave, garantindo que todos dependam exclusivamente da chave primária.
- III. A Terceira Forma Normal (3FN) busca eliminar dependências transitivas, evitando que atributos não chave dependam de outros atributos não chave.
- IV. A normalização elimina completamente a necessidade de junções entre tabelas, tornando as consultas sempre mais rápidas e simples.

Está correto apenas o que se afirma em:

d) I e III.

Desnormalização e Otimização no SQL Server

- A normalização garante integridade e consistência, mas cenários reais de produção, frequentemente, impõem requisitos de desempenho que exigem flexibilizações controladas. A desnormalização surge como abordagem complementar, aplicada de forma criteriosa para reduzir a complexidade das consultas, minimizar operações de junção e melhorar o tempo de resposta.
- Ferramentas como índices, planos de execução e técnicas de tuning permitem identificar gargalos e ajustar a estrutura do banco conforme os padrões reais de uso.

O Que É Desnormalização?

- A desnormalização consiste na introdução controlada de redundância em um modelo de dados previamente normalizado. Enquanto a normalização elimina redundâncias para garantir integridade, a desnormalização parte do princípio de que, em determinados cenários, a repetição de dados traz benefícios práticos – principalmente na redução do custo de junções em consultas frequentes.
- No SQL Server, operações de leitura intensiva são comuns em sistemas analíticos e aplicações de grande escala. A troca entre integridade estrutural e eficiência operacional precisa ser cuidadosamente avaliada.

Principal Motivação: Desempenho das Consultas

- Modelos Altamente Normalizados

Recuperação de informações exige múltiplas junções entre tabelas. O custo dessas operações cresce com o volume de dados e a complexidade das consultas.

Após a Desnormalização

Informações distribuídas em várias tabelas passam a ser armazenadas conjuntamente, reduzindo junções e resultando em consultas mais simples e rápidas.

- Menos operações de leitura em páginas distintas.
- Planos de execução mais simples.
- Ganhos expressivos em sistemas de alta leitura.

Vantagens da Desnormalização

Consultas Mais Rápidas

- Redução de junções gera planos de execução mais simples e eficientes, com menor consumo de CPU e memória.

Simplicidade de Acesso

- Consultas mais diretas facilitam o desenvolvimento e reduzem erros na lógica de acesso, beneficiando múltiplas equipes.

Escalabilidade de Leitura

- Menos junções significam menor sincronização entre threads, favorecendo escalabilidade horizontal e vertical.

Melhor Uso do Cache

- Dados concentrados em uma tabela aumentam a probabilidade de páginas em memória, reduzindo leituras físicas em disco.

Vantagens Práticas em Cenários Reais

- Em aplicações de relatórios gerenciais, painéis analíticos e sistemas de apoio à decisão, a carga de leitura é significativamente maior que a de escrita. A desnormalização pode reduzir expressivamente o tempo de resposta.
- **Planos de Execução Estáveis** - Menos junções produzem planos mais previsíveis, facilitando análise e otimização ao longo do tempo.
- **Índices Mais Eficientes** - Um único índice em tabela desnormalizada pode cobrir consultas complexas, reduzindo estruturas auxiliares a gerenciar.
- **Paralelismo Mais Efetivo** - Consultas simples se beneficiam de forma mais previsível do paralelismo, otimizando uso de múltiplos núcleos.

Observação Importante

- A organização dos dados em estruturas bem definidas reduz inconsistências e facilita a manutenção. Ao alterar essa organização em busca de desempenho, é necessário considerar os impactos na integridade das informações.
- A desnormalização é mais eficaz quando aplicada a conjuntos de dados relativamente estáveis e a consultas bem definidas, que justificam a introdução de redundância em troca de desempenho.

Desvantagens: Redundância e Integridade

- A principal desvantagem da desnormalização é o aumento da redundância de dados, que impacta diretamente a integridade e a consistência. Quando um mesmo dado é armazenado em múltiplos locais, qualquer alteração precisa ser propagada para todas as ocorrências.

Risco de Inconsistência

- Falhas na propagação de atualizações resultam em dados divergentes, comprometendo a confiabilidade do sistema.

Anomalias Transacionais

- Atualizações parciais ou falhas durante transações podem introduzir inconsistências semelhantes às de modelos não normalizados.

Integridade Referencial Enfraquecida

- Parte da integridade passa a depender de lógica na aplicação ou em procedures, aumentando o risco de erros.

Desvantagens: Operações de Escrita e Manutenção

Escrita Mais Custosa

- Uma única alteração precisa ser propagada para várias colunas ou registros, aumentando locks e consumo de recursos.

Maior Contenção

- Transações mais longas e complexas elevam o risco de bloqueios e deadlocks em ambientes de alta concorrência.

Manutenção Complexa

- Alterações estruturais exigem revisões cuidadosas em todas as ocorrências redundantes, elevando o custo de evolução do sistema.

Volume de Dados

- A repetição de informações aumenta o tamanho das tabelas, impactando backup, restauração e replicação.

Impacto na Indexação

- **Modelos Normalizados**
 - Índices mais específicos e direcionados.
 - Tabelas menores, menor custo de manutenção.
 - Estratégias de indexação mais simples.
- **Modelos Desnormalizados**
 - Tabelas mais largas com mais colunas.
 - Índices compostos extensos para diferentes padrões.
 - Cada escrita pode atualizar múltiplos índices.
 - Maior risco de degradação ao longo do tempo.
- A complexidade dos índices em modelos desnormalizados pode dificultar a escolha de estratégias adequadas e aumentar o risco de degradação de desempenho nas operações de escrita.

Uso com Moderação

- Embora a desnormalização possa melhorar o desempenho de consultas, seu uso excessivo pode aumentar a redundância e dificultar a manutenção do banco de dados. Por isso, essa estratégia deve ser aplicada apenas quando há necessidade comprovada de desempenho.
- Em muitos casos, problemas de desempenho podem ser resolvidos por meio de indexação adequada, reescrita de consultas ou uso de views indexadas, sem a necessidade de introduzir redundância estrutural.

Escalabilidade de Leitura vs. Escrita

Leitura: Benefícios

- Consultas com menos junções consomem menos CPU e memória.
- Mais consultas atendidas simultaneamente.
- Buffer pool mais eficiente com dados concentrados.

Escrita: Custos

- Tabelas maiores geram maior contenção de locks.
 - Inserções e atualizações exigem propagação para múltiplos registros.
 - Gargalos em sistemas com alta taxa de escrita.
-
- Esse contraste exige avaliação cuidadosa do perfil da carga de trabalho, considerando não apenas o volume atual, mas também a evolução esperada do sistema ao longo do tempo.

Desnormalização em Ambientes Distribuídos

- Em arquiteturas distribuídas ou com replicação, a desnormalização tem impactos significativos. O aumento da redundância implica maior volume de dados a replicar, elevando latência e consumo de largura de banda – especialmente em replicação síncrona.

Particionamento

- Dados alinhados à estratégia de particionamento geram ganhos; dependências cruzadas entre partições reduzem eficiência.

Cache de Aplicação

- Dados desnormalizados facilitam cache completo, mas a invalidação torna-se mais complexa com múltiplas entradas redundantes.

Alta Disponibilidade

- Maior volume de dados impacta backup, restauração e failover, afetando disponibilidade em cenários críticos.

Outros Riscos Críticos

Auditoria e Rastreamento

- Com informações em múltiplos locais, torna-se complexo identificar a fonte autoritativa e verificar consistência entre cópias – risco em ambientes regulados.

Perda de Flexibilidade

- Modelos altamente desnormalizados são específicos para padrões de consulta definidos, dificultando adaptação a novos requisitos em projetos de longo prazo.

Rigidez Arquitetural

- Alterações estruturais e mudanças nas regras de negócio exigem revisões profundas, elevando o custo de evolução em ambientes corporativos dinâmicos.

Abordagem Híbrida: O Melhor dos Dois Mundos

Modelo Normalizado

- Fonte primária de verdade para operações transacionais.
- Garante integridade referencial e facilita evolução do sistema.

Estruturas Desnormalizadas

- Utilizadas para leitura intensiva, relatórios e análises.
- Coexistem com o modelo principal, otimizando cada parte do sistema conforme suas necessidades.
- No SQL Server, essa abordagem pode ser implementada por meio de tabelas adicionais ou materializações controladas, mantendo equilíbrio entre desempenho e integridade.

Maturidade na Decisão de Desnormalizar

- Não se trata de escolher entre normalizar ou desnormalizar, mas de compreender quando cada abordagem é mais adequada e como combiná-las de forma equilibrada.
- Essa maturidade envolve conhecimento técnico do SQL Server, compreensão profunda do domínio do problema e sensibilidade para as necessidades do negócio. A desnormalização, quando bem aplicada, contribui para sistemas mais responsivos e escaláveis; quando aplicada sem planejamento, introduz problemas difíceis de corrigir.

Interatividade

Analise as afirmações:

- I. A desnormalização consiste na introdução controlada de redundância em um modelo de dados previamente normalizado, com o objetivo principal de reduzir o custo de junções em consultas frequentes.
- II. Em ambientes com alta taxa de escrita, a desnormalização tende a reduzir a contenção de locks e simplificar transações, tornando-se a estratégia mais indicada.
- III. Modelos desnormalizados podem apresentar ganhos significativos em desempenho de leitura, especialmente em cenários analíticos e de relatórios com consultas repetitivas.
- IV. A desnormalização elimina a necessidade de mecanismos adicionais de controle de integridade, pois a redundância garante consistência automática dos dados.

Interatividade

Está correto apenas o que se afirma em:

- a) I e II.
- b) II e IV.
- c) I, II e III.
- d) III e IV.
- e) I e III.

Resposta

Analise as afirmações:

- I. A desnormalização consiste na introdução controlada de redundância em um modelo de dados previamente normalizado, com o objetivo principal de reduzir o custo de junções em consultas frequentes.
- II. Em ambientes com alta taxa de escrita, a desnormalização tende a reduzir a contenção de locks e simplificar transações, tornando-se a estratégia mais indicada.
- III. Modelos desnormalizados podem apresentar ganhos significativos em desempenho de leitura, especialmente em cenários analíticos e de relatórios com consultas repetitivas.
- IV. A desnormalização elimina a necessidade de mecanismos adicionais de controle de integridade, pois a redundância garante consistência automática dos dados.

Está correto apenas o que se afirma em:

e) I e III.

Otimização como Processo Contínuo

- A otimização no SQL Server não deve ser encarada como uma atividade isolada ou pontual, mas como um processo contínuo que acompanha todo o ciclo de vida do sistema.
- Desde o projeto inicial do banco de dados até sua operação em produção, decisões relacionadas à estrutura das tabelas, à criação de índices e à forma como as consultas são escritas influenciam diretamente o comportamento do sistema.
- **O Tripé da Otimização**
 - **Índices** - Estruturas auxiliares que aceleram o acesso aos dados, reduzindo varreduras completas nas tabelas.
 - **Planos de Execução** - Estratégia escolhida pelo otimizador para executar cada consulta, revelando gargalos e custos.
 - **Tuning de Consultas** - Revisão e reescrita das instruções SQL para torná-las mais eficientes e alinhadas à estrutura dos dados.
- Essas três ferramentas não atuam de forma isolada – cada decisão tomada em uma delas impacta diretamente as demais.

Índices: A Base da Eficiência

- Os índices representam uma das ferramentas mais importantes de otimização no SQL Server. Eles funcionam como estruturas auxiliares que permitem ao mecanismo de banco de dados localizar registros de forma mais eficiente, reduzindo a necessidade de varreduras completas nas tabelas.
- Quando bem projetados, os índices reduzem significativamente o custo das operações de leitura. No entanto, cada índice ocupa espaço em disco e exige manutenção durante operações de inserção, atualização e exclusão – consumindo CPU, memória e I/O adicionais.

Tipos de Índices no SQL Server

Índices Clusterizados

- Determinam a forma física de armazenamento dos dados na tabela. Os dados são gravados na ordem das colunas que compõem o índice. Consultas com filtros ou intervalos baseados nessa chave tendem a ter desempenho significativamente melhor.

Índices Não Clusterizados

- Armazenados separadamente da tabela, contêm referências aos registros correspondentes. Úteis para consultas que filtram ou ordenam por colunas fora do índice clusterizado. O excesso pode gerar sobrecarga em operações de escrita.
- A escolha entre esses tipos influencia diretamente o desempenho das consultas e a organização interna do banco de dados.

Índices Avançados: Compostos e Incluídos

Índices Compostos

- Permitem indexar mais de uma coluna, úteis em consultas que filtram por múltiplos critérios.
- A ordem das colunas é fundamental – um projeto inadequado resulta em índices raramente utilizados.

Índices Incluídos

- Adicionam colunas ao índice sem que façam parte da chave de indexação. Permitem consultas cobertas – o SQL Server recupera todas as informações diretamente do índice, sem acessar a tabela base, reduzindo leituras. O uso excessivo, porém, aumenta o tamanho e o custo de manutenção.

Seletividade e Manutenção de Índices

Seletividade das Colunas

- Colunas com alta seletividade (grande variedade de valores distintos) são mais adequadas para indexação. Colunas com baixa seletividade podem levar o otimizador a preferir varreduras completas.

Fragmentação e Manutenção

- Operações de escrita causam fragmentação ao longo do tempo, reduzindo a eficiência dos índices. O SQL Server permite tratar isso via reorganização ou reconstrução, dependendo do grau de fragmentação.

Equilíbrio Leitura × Escrita

- A decisão de criar um índice deve considerar o padrão de acesso aos dados. Em ambientes com alta taxa de escrita, o custo de manutenção pode tornar-se um fator limitante para o desempenho.

O Que São Planos de Execução?

- Um plano de execução descreve, de forma detalhada, como o SQL Server pretende acessar os dados: quais operadores serão utilizados, a ordem das operações e as estimativas de custo associadas a cada etapa. Compreender esse plano permite identificar gargalos e orientar ajustes precisos.
- **Estimativas vs. Custos Reais**
 - **Plano Estimado**

Gerado com base em estatísticas disponíveis no momento da compilação. Reflete o que o otimizador acredita ser o caminho mais eficiente.
 - **Plano Real**

Reflete o comportamento efetivo durante a execução. Quando as estatísticas estão desatualizadas, o plano estimado pode divergir significativamente do real, resultando em planos subótimos.
- A atualização e a qualidade das estatísticas são elementos críticos para a eficácia dos planos de execução. Estatísticas imprecisas levam o otimizador a escolher métodos de acesso inadequados.

Identificando Gargalos nos Planos de Execução

Varreduras Completas de Tabelas

- Podem indicar ausência de índices adequados ou consultas que impedem o uso dos índices existentes. Em tabelas grandes, consomem grande quantidade de I/O e CPU.

Algoritmos de Junção Inadequados

- O SQL Server utiliza nested loops, merge join e hash join. A escolha errada para o volume real de dados resulta em desempenho insatisfatório.

Operações de Ordenação e Agregação

- Custosas especialmente em grandes volumes. Índices que forneçam dados já ordenados podem eliminar essas operações do plano.

Memória, Cache e Regressões de Desempenho

Pressão de Memória

- Operações como hash join e sort exigem grandes quantidades de memória. Quando insuficiente, o SQL Server recorre ao disco, aumentando significativamente o tempo de execução.

Reutilização de Planos

- O SQL Server armazena planos em cache para reutilização. Porém, planos reutilizados podem não ser ideais para diferentes parâmetros de entrada, gerando desempenho inconsistente.

Prevenção de Regressões

- Alterações no esquema, nos índices ou nas consultas podem degradar planos existentes.
- Comparar planos antes e depois de alterações permite identificar mudanças indesejadas antes que afetem a produção.

Tuning: Diagnóstico em Ação

- O tuning de consultas transforma diagnósticos em ações concretas de melhoria. Envolve a revisão cuidadosa da forma como as consultas são escritas, considerando não apenas o resultado esperado, mas o caminho percorrido pelo mecanismo de banco de dados para produzi-lo.
- **Princípios Fundamentais do Tuning**
 - **Reduzir o Volume de Dados Processados**
Aplicar filtros o mais cedo possível no plano de execução, restringindo o número de linhas nas etapas subsequentes.
 - **Revisar Junções**
Avaliar se todas as junções são necessárias e se podem ser reestruturadas.
 - **Evitar Funções em Colunas Filtradas**
Funções aplicadas diretamente a colunas em cláusulas de filtro impedem o uso de índices.
 - **Padronizar Tipos de Dados**
Comparações entre tipos incompatíveis exigem conversões implícitas, aumentando o custo.

Agregações e Otimização de Escrita

Otimizando Agregações

- Operações de agregação podem ser custosas em grandes volumes. Avaliar se podem ser realizadas em etapas anteriores ou restritas a subconjuntos menores é uma estratégia eficaz. Índices que suportem as agregações podem eliminar operações adicionais de ordenação ou hashing.

Visão Sistêmica das Otimizações

- Uma otimização que melhora o desempenho de uma consulta específica pode aumentar a carga sobre o servidor ou afetar negativamente outras consultas. As decisões de tuning devem ser avaliadas de forma sistêmica, considerando o impacto global sobre o ambiente SQL Server.

Monitoramento Contínuo

- O comportamento das consultas pode mudar ao longo do tempo, à medida que o volume de dados cresce ou que os padrões de acesso se modificam. Ferramentas de monitoramento permitem identificar consultas que passaram a consumir mais recursos do que o esperado, fornecendo subsídios para novas intervenções de tuning.
- Consultas executadas com alta frequência merecem atenção especial – qualquer melhoria em seu desempenho pode gerar ganhos cumulativos significativos.

Uma Abordagem Integrada e Madura

- **Índices com Critério** - Nem ausência nem excesso. A maturidade está em reconhecer quando um índice agrega valor e quando se torna um ônus.
- **Planos como Evidência** - Transformam a otimização em atividade baseada em dados observáveis, não em intuição.
- **Tuning Sistemático** - Estabelece padrões de escrita que favorecem a eficiência de forma consistente e documentada.
- **Processo Permanente** - Monitoramento, análise e ajuste contínuos – não apenas em momentos de crise.
- A otimização deve ser entendida como parte integrante da engenharia de bancos de dados, acompanhando todas as fases do ciclo de vida do sistema e contribuindo para soluções que funcionam com eficiência, confiabilidade e capacidade de evolução.

Interatividade

Analise as afirmações:

- I. Índices bem projetados reduzem a necessidade de varreduras completas de tabelas, melhorando o desempenho das operações de leitura, porém aumentam o custo das operações de escrita devido à necessidade de manutenção.
- II. Planos de execução refletem exclusivamente o comportamento real da consulta em tempo de execução, não sendo influenciados por estatísticas ou estimativas do otimizador.
- III. A aplicação de funções diretamente em colunas utilizadas em filtros favorece o uso de índices e melhora o desempenho das consultas.
- IV. O tuning de consultas envolve a reescrita e revisão das instruções SQL, buscando reduzir o volume de dados processados e facilitar a escolha de planos de execução mais eficientes.

Interatividade

Está correto apenas o que se afirma em:

- a) II e IV.
- b) I, II e III.
- c) II e III.
- d) I e IV.
- e) I e III.

Resposta

Analise as afirmações:

- I. Índices bem projetados reduzem a necessidade de varreduras completas de tabelas, melhorando o desempenho das operações de leitura, porém aumentam o custo das operações de escrita devido à necessidade de manutenção.
- II. Planos de execução refletem exclusivamente o comportamento real da consulta em tempo de execução, não sendo influenciados por estatísticas ou estimativas do otimizador.
- III. A aplicação de funções diretamente em colunas utilizadas em filtros favorece o uso de índices e melhora o desempenho das consultas.
- IV. O tuning de consultas envolve a reescrita e revisão das instruções SQL, buscando reduzir o volume de dados processados e facilitar a escolha de planos de execução mais eficientes.

Está correto apenas o que se afirma em:

d) I e IV.

Referências

- ALVES, William P. *Banco de dados: teoria e desenvolvimento*. Rio de Janeiro: Érica, 2021. *E-book*.
- DATE, C. J. *Introdução a sistemas de bancos de dados*. Rio de Janeiro: LTC, 2023.
- ELMASRI, R.; NAVATHE, S. B. *Sistemas de banco de dados*. 7. ed. São Paulo: Pearson, 2018. *E-book*.
- HEUSER, Carlos Alberto. *Projeto de bancos de dados*. Porto Alegre: Bookman, 2009.
- MACHADO, Felipe Nery R. *Banco de dados – projeto e implementação*. Rio de Janeiro: Érica, 2020. *E-book*.
- PICHETTI, Roni F.; VIDA, Edinilson S.; CORTES, Vanessa S. M P. *Banco de dados*. Porto Alegre: SAGAH, 2021. *E-book*.
- SILBERSCHATZ, Abraham. *Sistema de banco de dados*. Rio de Janeiro: GEN LTC, 2020. *E-book*.
- SILVA, Luiz F C.; RIVA, Aline D.; ROSA, Gabriel A. *et al. Banco de dados não relacional*. Porto Alegre: SAGAH, 2021. *E-book*.

ATÉ A PRÓXIMA!